

ALWAYS ON

Final Architecture, Isolation, Implementation, and Operations Report

1. Purpose and Design Principles

ALWAYS ON is an on-premises operational platform for drone telemetry, mapping, vehicle simulation, fabrication/facility simulation, customer sales support, AI-assisted community interaction, and cryptographically verifiable business/data provenance.

The platform runs on a Kubuntu 26 LTS-class workstation with an AMD CPU and EVGA NVIDIA GTX 1080 GPU. It uses **Podman only**, managed declaratively with systemd Quadlet units. Quadlet generates systemd services from container, network, volume, pod, and related unit definitions, avoiding fragile shell wrappers and manually generated unit files. [redhat](#)

The central design requirement is **hard functional separation**. Sales and AI services must not reach flight control. Mapping must not reach payment data. Simulation must not reach live hardware. Corda must not become a bulk-data store or a real-time control plane.

Core principles

- The pCloud-hosted static storefront is separate from the local operational host.
- Public traffic never directly reaches drone, simulation, mapping, database, AI, or Corda services.
- Every major workload has a separate Podman network, storage path, service identity, database role, and secrets scope.
- Corda records receipts, authorization, ownership, entitlement, approval, and provenance manifests—not raw high-rate telemetry, imagery, payment details, or large simulation files.
- pCloud and IPFS are replication/distribution layers, not the primary operational datastore.
- No container uses `--privileged` unless a specific, documented hardware requirement cannot be met through a narrow device mapping.
- GPU-heavy workloads are scheduled and resource-limited so they do not degrade field telemetry, mapping, databases, or core operations.
- Containers use pinned image digests for critical workloads. Automatic updates are restricted to low-risk services and require tested rollback procedures.

2. Platform Baseline

Area	Selected baseline
Host	Kubuntu 26 LTS-class workstation
Host compute	AMD CPU, EVGA NVIDIA GTX 1080
Container runtime	Podman; no Docker daemon, no Watchtower
Service management	systemd with Podman Quadlet
Host networking	Netplan; UFW or nftables default-deny inbound policy
Public storefront	Static HTML in pCloud Public Folder
Public customer community	Mastodon, linked from storefront
AI support	OpenClaw agent plus internal LM Studio Core/headless service
Sales payments	Hosted payment-provider checkout; credit card, PayPal, optional wire-transfer workflow
Field computer	Raspberry Pi 5 with Waveshare SX1262-class LoRa HAT/top-hat
Desktop RF endpoint	Heltec WiFi LoRa 32 V3 over USB serial
Drone stack	ArduPilot, MAVLink, RNS/Reticulum, MeshChatX, LoRa
Mapping	WebODM, NodeODM/processing worker, queue, database, persistent object/filesystem storage

Area	Selected baseline
Vehicle simulation	ROS 2 Jazzy, Gazebo Harmonic, ArduPilot SITL, MAVLink, QGroundControl, SB3
Fabrication simulation	ROS 2 Jazzy, Gazebo Harmonic, robot arms, additive manufacturing, storage, assembly, pantry/fridge/kitchen, transfer systems
Ledger	Corda through a dedicated ledger-ingestion gateway
Backup	Local backup plus encrypted off-host replication with restore testing
Archive/provenance	Local source data, signed manifests, Corda receipts, encrypted pCloud archives, controlled IPFS CIDs

WebODM officially documents a Podman-based alternative deployment path, including use of Podman and its Docker compatibility layer. [docs.webodm](#)

ROS 2 Jazzy and Gazebo Harmonic are the selected simulation baseline. ROS documentation includes Gazebo simulation guidance for Jazzy and the Jazzy package ecosystem supports the `ros_gz` integration path. [docs.ros](#)

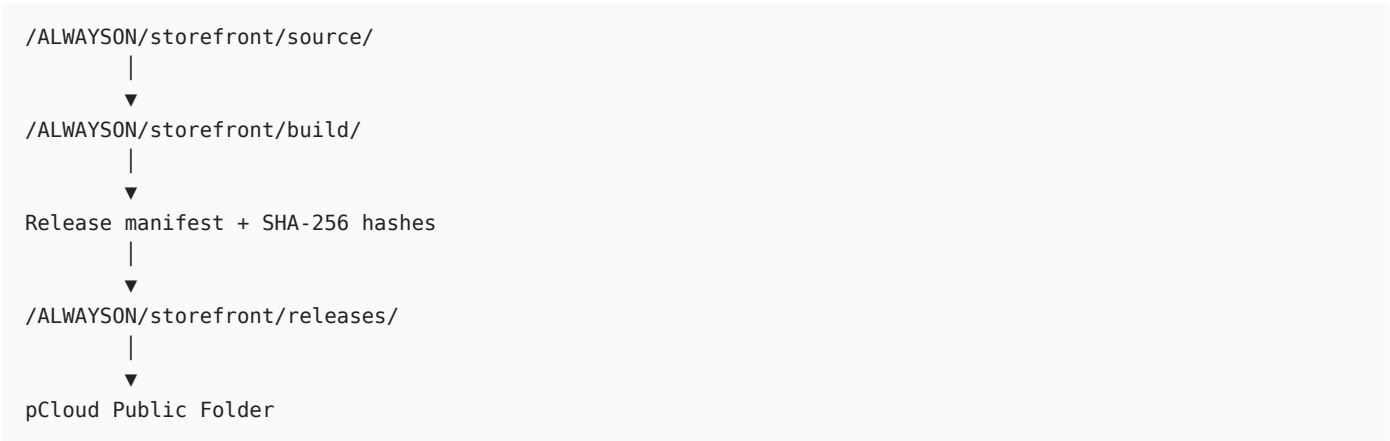
3. Public Storefront and Sales

3.1 pCloud static HTML storefront

The public site is hosted in the **pCloud Public Folder**, not on the Kubuntu host. pCloud documents that its Public Folder can create direct links, host static HTML websites, and embed public content. [help.pcloud](#)



The public storefront source is developed locally, versioned, signed, built, and released to pCloud:



3.2 Public-site restrictions

The static site must never contain:

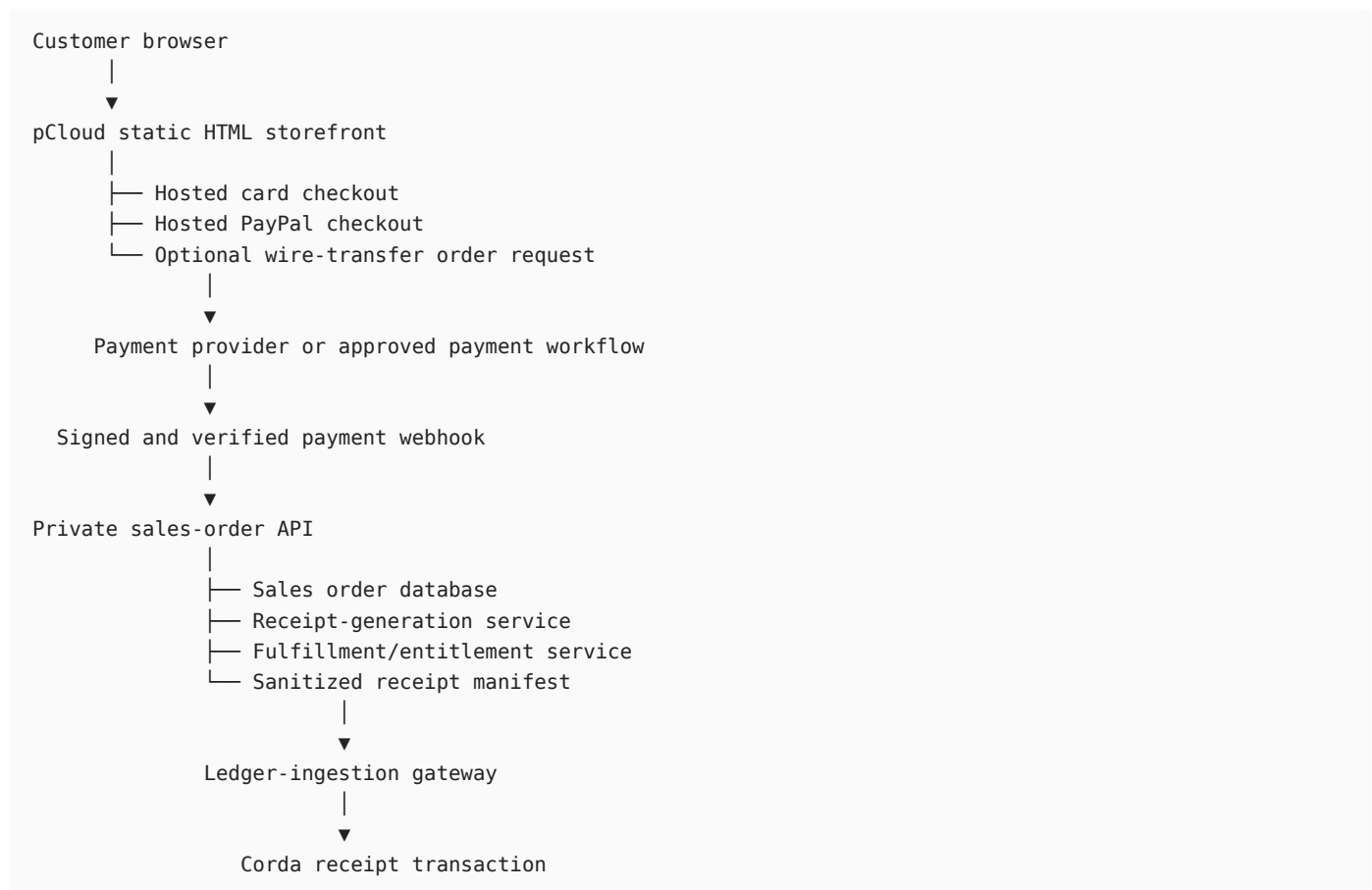
- Private API keys.

- Corda node or RPC credentials.
- Payment-provider secret keys.
- Mastodon OAuth tokens.
- pCloud credentials.
- IPFS private-swarm credentials.
- Local host IP addresses.
- Local service ports.
- Drone, field, mapping, or simulation endpoints.
- Database connection details.
- Customer order data.

Client-side JavaScript may contain only public configuration, such as a payment-provider publishable key if required by the provider, approved Mastodon links, documentation URLs, and a public support route.

3.3 Sales and payment workflow

3DR's published customer-order guidance indicates immediate processing for credit-card and PayPal payments, with wire transfers handled as a separate process after funds are received. The equivalent ALWAYS ON workflow is: [docs.3dr](#)



3.4 Payment security rules

- Use a hosted checkout or provider-hosted payment page.
- Do not collect or store card numbers, CVV values, PayPal credentials, payment-session secrets, or bank credentials.
- Verify payment-provider webhooks using signature verification, replay prevention, timestamp tolerance, and idempotency keys.
- Retain only the provider transaction reference or a salted hash/reference where practical.
- Keep payment-webhook verification separate from Mastodon, OpenClaw, LM Studio, WebODM, simulation, field services, and Corda core.
- Corda receives only an approved receipt/order/entitlement manifest, never raw payment payloads.

4. Mastodon, OpenClaw, and LM Studio

The public storefront links users to Mastodon for order follow-up, product discussion, community interaction, and announcements. It may also link to an AI-assisted support interface.

Mastodon is a **communications channel**, not the authoritative order-management or payment system.

4.1 Permitted AI functions

OpenClaw and LM Studio may:

- Draft product and documentation answers.
- Summarize public community threads.
- Help users locate product pages, manuals, and support information.
- Draft Mastodon posts for operator approval.
- Classify incoming support requests.
- Create internal summaries for staff.
- Generate responses from a curated, non-sensitive knowledge base.

4.2 Prohibited AI functions

OpenClaw and LM Studio must not:

- Access payment-provider credentials or customer payment data.
- Access unredacted sales/order database records.
- Initiate refunds, charge customers, or create payment instructions.
- Alter customer order status.
- Submit Corda transactions.
- Read or command MAVLink, QGroundControl, ArduPilot, ROS, Gazebo, or LoRa systems.
- Access WebODM workers, imagery staging, or raw mapping results.
- Receive secrets, private keys, API tokens, or raw customer PII in prompts.
- Automatically publish financial, legal, safety, delivery, or technical commitments without human approval.

4.3 AI containment

```
ao-sales network
├─ Mastodon integration
├─ OpenClaw
├─ LM Studio Core
├─ Curated support knowledge base
└─ Approval queue

OpenClaw
├─ May call LM Studio internally
├─ May call Mastodon API using limited OAuth scope
├─ May create draft support records
└─ Cannot route to payment, field, mapping, simulation, or Corda networks
```

The GTX 1080 may be assigned to LM Studio only after GPU passthrough is proven on the actual Podman/NVIDIA driver configuration. Set hard memory, CPU, GPU, concurrency, and restart limits. Treat the GPU as a shared resource.

5. Isolation Domains

The system uses separate Podman networks, volumes, service accounts, credentials, and data paths.

ao-sales
 ao-payment
 ao-field
 ao-mapping
 ao-sim-vehicle
 ao-sim-fabrication
 ao-ledger-ingest
 ao-ledger-core
 ao-data
 ao-admin

5.1 Domain matrix

Domain	Included services	Public access	Direct Corda access	Direct hardware access
Sales	Order API, Mastodon integration, OpenClaw, LM Studio, receipt generator	No	No	No
Payment	Webhook verifier, payment-provider adapter	No	No	No
Field	Heltec gateway, RNS, MeshChatX, telemetry store, mission-release service	No	No	Heltec USB serial only
Mapping	WebODM, NodeODM, Redis, mapping DB, imagery intake	VPN/operator only	No	Optional GTX 1080 only after validation
Vehicle simulation	ROS 2, Gazebo, ArduPilot SITL, MAVLink, QGroundControl simulation, SB3	No	No	No live vehicle/radio hardware
Fabrication simulation	ROS 2, Gazebo, robot/facility models, task scheduler	No	No	No live fabrication/kitchen hardware initially
Ledger ingestion	API gateway, validation, mTLS, signature verification, event audit	No	Yes, controlled	No
Ledger core	Corda node, Corda DB, certificate material	No	Internal only	No
Archive	pCloud replication, private/encrypted IPFS staging and pinning	No	Manifest receipt only	No
Administration	Monitoring, logs, backups, VPN-only access	VPN only	Controlled administrative access	Limited host administration only

5.2 Prohibited paths

Sales → MAVLink, ROS, Gazebo, LoRa, RNS, MeshChatX
 Sales → WebODM worker or raw imagery store
 Sales → Corda core

 Payment → Mastodon, OpenClaw, LM Studio
 Payment → field, simulation, mapping, Corda core

 Field → payment provider, Mastodon, OpenClaw, LM Studio
 Field → mapping workers or Corda core

 Mapping → drone command/control, payment provider, Mastodon
 Mapping → Corda core

Vehicle simulation → live drone, LoRa/RNS, sales, payment, Corda core
Fabrication simulation → live printer/LPBF/arms/kitchen hardware, sales, payment, Corda core

Public Internet → databases, Redis, LM Studio, Corda, WebODM workers,
RNS, MeshChatX, MAVLink, ROS, Gazebo, QGroundControl

5.3 Permitted cross-domain paths

Sales receipt manifest → Ledger-ingestion gateway
Field telemetry manifest → Ledger-ingestion gateway
Mapping deliverable manifest → Ledger-ingestion gateway
Vehicle simulation manifest → Ledger-ingestion gateway
Fabrication simulation manifest → Ledger-ingestion gateway

Ledger receipt/approval state → Authorized domain API only
Signed mission release → Field mission-release service
Approved imagery upload → Mapping staging service

Each cross-domain request requires:

- Mutual TLS.
- Dedicated client certificate or service identity.
- Domain-specific authorization.
- Signed request body.
- Timestamp and nonce.
- Idempotency key.
- Versioned schema.
- Payload hash.
- Audit event.
- Rate limiting.
- Explicit allowlist route.

6. Hardware and Radio Design

6.1 Drone-side hardware

```
ArduPilot flight controller
|
| MAVLink over UART or USB
▼
Raspberry Pi 5
├─ MAVLink collection / mission agent
├─ Local encrypted telemetry spool
├─ RNS / Reticulum node
├─ MeshChatX service
├─ Packet signing and acknowledgement logic
└─ Waveshare SX1262-class LoRa HAT
    |
    ▼
    LoRa RF link
```

The Waveshare LoRa HAT family uses SX1262/SX1268-class radio hardware and is commonly documented as a serial LoRa module for Raspberry Pi integration. [waveshare](https://www.waveshare.com/)

6.2 Desktop-side radio hardware

```

Heltec WiFi LoRa 32 V3
| USB-C serial
▼
/dev/serial/by-id/...
|
▼
Dedicated Heltec gateway service
├─ Serial framing
├─ Packet authentication
├─ Replay detection
├─ Raw-packet persistence
├─ RNS / MeshChatX adapter
├─ Link-quality metrics
├─ Telemetry normalization
└─ Signed telemetry-manifest exporter

```

The Heltec V3 uses an ESP32-S3 and SX1262 LoRa radio, with USB serial capability and regional variants. [heltec](#)

6.3 LoRa configuration requirements

Both endpoints must be confirmed as appropriate US915 hardware variants and configured consistently.

Create the following versioned configuration files:

```

/ALWAYS0N/config/drone/waveshare-lora/radio-profile-us915.yaml
/ALWAYS0N/config/field/heltec-v3/radio-profile-us915.yaml

```

Each must contain:

- Exact hardware SKU and serial number.
- Regulatory region.
- Frequency/channel plan.
- Bandwidth.
- Spreading factor.
- Coding rate.
- Preamble length.
- Transmit power.
- Sync word or network identifier.
- Maximum packet size.
- Airtime limits and retry policy.
- Encryption policy.
- Device public key.
- Key rotation policy.
- Sequence-number policy.
- Replay-protection policy.
- Acknowledgement and retransmission rules.
- Message priority classes.
- Firmware version and configuration hash.

Do not describe the system as LoRaWAN unless it uses LoRaWAN-compliant device behavior, gateway forwarding, a network server, join/security procedures, and an appropriate regional channel plan. The proposed RNS/MeshChatX transport over LoRa is a separate design.

6.4 Hardware-access policy

The Heltec gateway container receives only its designated serial device, preferably by stable path:

```
/dev/serial/by-id/usb-Silicon_Labs_CP2102_...
```

Avoid generic `/dev/ttyUSB0` mappings where possible. Do not use `--privileged` as the normal deployment method. If the service cannot operate rootlessly with a narrow device mapping and group permissions, isolate it as a small, explicitly documented host-level or rootful gateway service.

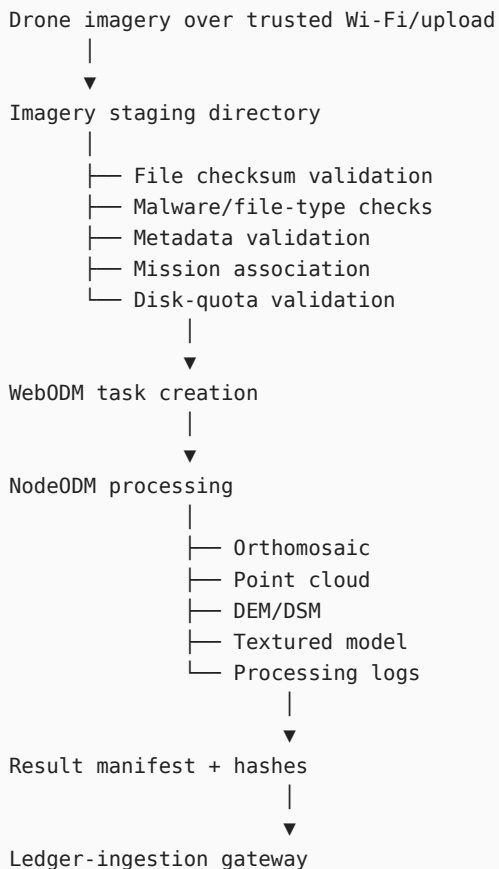
7. Mapping: WebODM Under Podman

WebODM is deployed entirely in the isolated mapping domain. WebODM's documentation recognizes Podman as an alternative to Docker, using Podman and the Docker-compatibility layer where required by the project tooling. [docs.webodm](https://docs.webodm.com)

7.1 Mapping components

```
ao-mapping
├── imagery-ingest
├── webodm-web
├── webodm-worker
├── nodeodm
├── mapping-postgres
├── mapping-redis
├── mapping-result-exporter
└── mapping-health-monitor
```

7.2 Mapping data flow



7.3 Persistent paths


```
/ALWAYSON/data/mapping/incoming/  
/ALWAYSON/data/mapping/validated/  
/ALWAYSON/data/mapping/rejected/  
/ALWAYSON/data/mapping/webodm-media/  
/ALWAYSON/data/mapping/nodeodm/  
/ALWAYSON/data/mapping/deliverables/  
/ALWAYSON/artifacts/mapping-manifests/
```

7.4 GPU policy

Start WebODM in CPU-only mode. Validate full processing, recovery, disk usage, and output integrity before adding GPU acceleration.

If GPU use is validated:

- Assign GPU access only to the specific NodeODM/processing workload that benefits.
- Do not expose GPU devices to PostgreSQL, Redis, Corda, RNS, MeshChatX, or gateway services.
- Limit concurrent GPU tasks.
- Schedule LM Studio and WebODM GPU work to prevent VRAM contention.
- Keep one tested CPU-only recovery path.
- Record driver version, CUDA/runtime version, image digest, and processing profile in each mapping manifest.

8. Vehicle Simulation

The vehicle simulator is a self-contained development/test environment.

```
ao-sim-vehicle  
├─ ROS 2 Jazzy  
├─ Gazebo Harmonic  
├─ ArduPilot SITL  
├─ ROS-Gazebo bridge  
├─ MAVLink router  
├─ QGroundControl simulation client  
├─ Vehicle and sensor models  
├─ Mission/scenario runner  
├─ Optional Stable-Baselines3 jobs  
├─ Result exporter  
└─ Scenario/version registry
```

8.1 Vehicle-simulation capabilities

- Multirotor, VTOL, biplane, fixed-wing, or ground vehicle models.
- Terrain, obstacles, wind, weather, takeoff/landing zones, and mission geofencing.
- GPS, IMU, barometer, camera, rangefinder, and telemetry simulation.
- MAVLink mission protocol and telemetry behavior.
- Battery depletion, motor failure, sensor drift, degraded GPS, communications loss, and emergency-return scenarios.
- Software-in-the-loop ArduPilot validation.
- Optional autonomy-policy evaluation.

8.2 Vehicle-simulation separation

```
ROS_DOMAIN_ID=21  
GZ_PARTITION=alwayson_vehicle_sim
```

These are example values. Final identifiers must be tracked in the architecture register.

Rules:

- No live flight controller connections.
- No live LoRa/RNS/MeshChatX hardware.
- No production credentials.
- No public ports.
- No raw customer or payment data.
- No direct Corda submission.
- Only a dedicated exporter may submit approved, signed run manifests to the ledger-ingestion gateway.

8.3 Vehicle result manifest

Each run produces:

- Simulation run ID.
- Scenario ID and world hash.
- Vehicle model hash.
- ArduPilot build version/hash.
- ROS/Gazebo image digests.
- Mission/configuration hash.
- Start/end time.
- Result state.
- Output hashes.
- Human/operator approval state.
- Exporter signature.

9. Fabrication and Kitchen Simulation

The fabrication environment is separate from vehicle simulation and represents a production/facility workflow. It initially simulates logical operation, robotics motion, material flow, task sequencing, inventory movement, and safety interlocks. It does **not** control live production hardware in the first implementation phase.

```
ao-sim-fabrication
├─ ROS 2 Jazzy
├─ Gazebo Harmonic
├─ Robot-arm cells
├─ 3D-printer cell
├─ LPBF process-area model
├─ Storage and inventory model
├─ Assembly stations
├─ Refrigerator model
├─ Pantry model
├─ Kitchen model
├─ Carousels / conveyors
├─ Pass-through ports
├─ Material-handling arms
├─ Facility task scheduler
├─ Safety/interlock model
└─ Fabrication-result exporter
```

9.1 Facility flow

```
Storage
|
▼
```

```

Carousel / conveyor
|
▼
Robot-arm pickup
|
├─ 3D printer cell
├─ LPBF cell
├─ Assembly station
├─ Refrigerator / pantry
└─ Kitchen / pass-through

```

9.2 Fabrication simulation goals

- Validate robot-arm reachability, collision avoidance, and transfer paths.
- Evaluate carousel, conveyor, and pass-through sequencing.
- Model storage allocation, inventory routing, and queue constraints.
- Test handoff rules between additive manufacturing, assembly, storage, and food-related areas.
- Model safety zones, restricted zones, motion interlocks, and fail-safe behavior.
- Test task schedules, capacity, bottlenecks, and material movement.
- Create reproducible facility process manifests.

9.3 Fabrication isolation

```

ROS_DOMAIN_ID=22
GZ_PARTITION=alwayson_fabrication_sim

```

This simulation has:

- No live 3D-printer, LPBF, robot-arm, refrigeration, carousel, or kitchen-device control during phase one.
- No access to vehicle simulation.
- No access to sales, payment, Mastodon, AI, field, mapping, or Corda core.
- Only a signed exporter path to the ledger-ingestion gateway.

9.4 Safety boundary

LPBF and food/kitchen operations have fundamentally different contamination, safety, thermal, powder, and operational risks. The simulation should represent them as separate safety zones and should not imply co-location, shared air handling, shared transfer paths, or shared tooling unless a real facility safety design has validated that arrangement.

10. Corda, Data Provenance, pCloud, and IPFS

Corda is used as a **private receipt, provenance, ownership, approval, and entitlement ledger**. Corda's design emphasizes transaction privacy among relevant parties rather than globally broadcasting transaction data. [github](#)

10.1 Corda architecture

```

Every domain
|
▼
Signed manifest/event
|
▼
ao-ledger-ingest
├─ Mutual TLS
├─ Schema validation
└─ Signature verification

```

```

├─ Authorization enforcement
├─ Idempotency handling
├─ Audit logging
├─ Corda transaction submission
│
└─
    ▼
  ao-ledger-core
  ├─ Corda node
  ├─ Corda database
  ├─ Certificate materials
  ├─ Notary/network configuration as applicable
  └─ Backup/restore process

```

No service outside `ao-ledger-ingest` receives direct access to the Corda node or Corda database.

10.2 What belongs in Corda

Object	Corda stores	External storage stores
Sales receipt	Order ID, product, status, provider reference hash, entitlement, fulfillment state, signature	Full invoice, raw payment webhooks, shipping information, customer PII
Telemetry batch	Device ID, mission ID, time window, batch hash, quality state, authorization	Raw telemetry, radio metadata, high-rate streams
Mapping job	Mission ID, source hash, processing-profile hash, deliverable hashes, ownership/license state	Imagery, GeoTIFF, LAZ, OBJ, point clouds, WebODM files
Vehicle simulation	Scenario/model/software hashes, approval state, result hash	ROS bags, MAVLink logs, world assets, SB3 weights
Fabrication simulation	Facility scenario hash, robot/task plan hash, result hash, approval state	Robot trajectories, facility logs, models, render assets
Product release	Firmware/software/image hashes, signer, release state	Container layers, build artifacts, source tree
User/customer	Pseudonymous identity/consent/entitlement reference	Names, emails, addresses, support content, payment data

10.3 Manifest schema

Each external artifact receives a signed manifest before Corda submission:

```

{
  "object_id": "UUID",
  "object_type": "sales_receipt | telemetry_batch | map_product | vehicle_simulation |
fabrication_simulation",
  "origin_domain": "sales | field | mapping | sim_vehicle | sim_fabrication",
  "created_at_utc": "ISO-8601",
  "schema_version": "1.0",
  "content_hash_sha256": "HEX_DIGEST",
  "content_size_bytes": 0,
  "local_storage_reference": "opaque internal reference",
  "ipfs_cid": "optional encrypted artifact CID",
  "pcloud_archive_reference": "optional opaque encrypted archive reference",
  "authorization_policy_id": "policy identifier",
  "producer_key_id": "service identity key ID",
  "signature": "detached signature"
}

```

Corda receives the manifest hash, relevant business state, signer identity, permissions, approval state, transaction ID, and receipt result.

10.4 pCloud and IPFS policy

```
Local authoritative data
├──
├── SHA-256 hash
├── Signed manifest
├── Corda receipt
├── Encrypted pCloud archive
└── Private/encrypted IPFS replication
```

Rules:

- Local /ALWAYSON/data/ remains the primary operational store.
- Encrypt data before pCloud replication unless the storage and access model has been formally approved.
- Do not place customer PII, payment data, private keys, raw telemetry, sensitive imagery, or proprietary design files on public IPFS.
- Use a private IPFS swarm, controlled pinning, or encryption-before-upload for sensitive material.
- Store both SHA-256 hash and IPFS CID when IPFS is used.
- Store CIDs and encrypted archive references in Corda, not bulk data.
- Verify periodic restore from pCloud/archive and local backup media.

11. Root Filesystem Structure

```
/ALWAYSON/
├── README.md
├── VERSION
├──
├── docs/
├──   ├── adr/
├──   │   ├── ADR-001-domain-isolation.md
├──   │   ├── ADR-002-pcloud-static-storefront.md
├──   │   ├── ADR-003-payment-processing.md
├──   │   ├── ADR-004-corda-provenance.md
├──   │   ├── ADR-005-pcloud-ipfs-policy.md
├──   │   ├── ADR-006-field-radio-architecture.md
├──   │   ├── ADR-007-webodm-podman.md
├──   │   ├── ADR-008-vehicle-simulation.md
├──   │   └── ADR-009-fabrication-simulation.md
├──   ├── architecture/
├──   │   ├── domain-boundaries.md
├──   │   ├── network-diagram.md
├──   │   ├── port-register.md
├──   │   ├── data-flow.md
├──   │   ├── cross-domain-api-contracts.md
├──   │   └── data-classification.md
├──   ├── runbooks/
├──   │   ├── backup-restore.md
├──   │   ├── disaster-recovery.md
├──   │   ├── corda-recovery.md
├──   │   ├── pcloud-site-publish.md
├──   │   ├── podman-update-rollback.md
├──   │   ├── field-gateway-recovery.md
├──   │   ├── webodm-recovery.md
├──   │   └── simulation-recovery.md
├──   └── compliance/
├──       ├── privacy.md
├──       ├── retention.md
├──       ├── payment-data-handling.md
├──       └── access-control.md
```

```
|
|— storefront/
|   |— source/
|   |   |— index.html
|   |   |— products/
|   |   |— catalog/
|   |   |— support/
|   |   |— community/
|   |   |— legal/
|   |   |— assets/
|   |— build/
|   |— releases/
|   |— manifests/
|   |— pcloud-public-folder/
|
|— quadlet/
|   |— networks/
|   |   |— ao-sales.network
|   |   |— ao-payment.network
|   |   |— ao-field.network
|   |   |— ao-mapping.network
|   |   |— ao-sim-vehicle.network
|   |   |— ao-sim-fabrication.network
|   |   |— ao-ledger-ingest.network
|   |   |— ao-ledger-core.network
|   |   |— ao-data.network
|   |   |— ao-admin.network
|   |— volumes/
|   |— sales/
|   |— payment/
|   |— field/
|   |— mapping/
|   |— sim-vehicle/
|   |— sim-fabrication/
|   |— ledger/
|   |— operations/
|
|— config/
|   |— storefront/
|   |— sales/
|   |— payment/
|   |— field/
|   |   |— heltec-v3/
|   |   |— rns/
|   |   |— meshchatx/
|   |   |— telemetry-schema/
|   |— drone/
|   |   |— rpi5/
|   |   |— waveshare-lora/
|   |   |— ardupilot/
|   |   |— rns/
|   |   |— meshchatx/
|   |— mapping/
|   |   |— webodm/
|   |   |— processing-profiles/
|   |— sim-vehicle/
|   |   |— ros2-jazzy/
|   |   |— gazebo-harmonic/
|   |   |— ardupilot/
|   |   |— qgroundcontrol/
|   |   |— missions/
|   |— sim-fabrication/
|   |— ros2-jazzy/
```

```
| | | gazebo-harmonic/
| | | robot-arms/
| | | 3d-printer/
| | | lpb/
| | | storage/
| | | assembly/
| | | fridge-pantry/
| | | kitchen/
| | | carousels/
| | | passthroughs/
| | ledger/
| | | corda/
| | | contracts/
| | | flows/
| | | schemas/
| | | certificates/
| | pcloud/
| | ipfs/
| secrets/
| | README.md
| | .gitignore
| data/
| | sales/
| | payment/
| | field/
| | | raw-telemetry/
| | | normalized-telemetry/
| | | gateway-spool/
| | | device-registry/
| | | mission-cache/
| | mapping/
| | | incoming/
| | | validated/
| | | rejected/
| | | webodm-media/
| | | nodeodm/
| | | deliverables/
| | sim-vehicle/
| | | worlds/
| | | scenarios/
| | | ros-bags/
| | | mavlink-logs/
| | | sb3-models/
| | | exports/
| | sim-fabrication/
| | | worlds/
| | | robot-trajectories/
| | | material-flow/
| | | printer-jobs/
| | | lpb-jobs/
| | | inventory/
| | | assembly-runs/
| | | kitchen-runs/
| | | safety-events/
| | | exports/
| | ledger/
| | cache/
| artifacts/
| | storefront-releases/
| | drone-releases/
```

```
| |─ telemetry-manifests/
| |─ mapping-manifests/
| |─ vehicle-simulation-manifests/
| |─ fabrication-simulation-manifests/
| |─ sales-receipts/
|
|─ ipfs/
| |─ staging/
| |─ manifests/
| |─ pinning-queue/
| |─ approved-cids/
|
|─ pcloud/
| |─ storefront-deploy/
| |─ encrypted-archive-staging/
| |─ replication-manifests/
| |─ restore-cache/
|
|─ backups/
| |─ databases/
| |─ corda/
| |─ configurations/
| |─ storefront/
| |─ mapping/
| |─ simulation/
| |─ restic/
| |─ restore-tests/
|
|─ logs/
| |─ sales/
| |─ payment/
| |─ field/
| |─ mapping/
| |─ sim-vehicle/
| |─ sim-fabrication/
| |─ ledger/
| |─ security/
|
|─ scripts/
| |─ bootstrap/
| |─ deploy/
| |─ storefront/
| |─ radio/
| |─ mapping/
| |─ simulation/
| |─ ledger/
| |─ backup/
| |─ restore/
| |─ validation/
|
|─ tests/
| |─ unit/
| |─ integration/
| |─ security/
| |─ load/
| |─ disaster-recovery/
|
|─ tmp/
| |─ storefront-build/
| |─ upload-quarantine/
| |─ webodm-processing/
| |─ simulation-work/
```

12. Security Controls

12.1 Firewall policy

Only public HTTPS services, if any are intentionally hosted locally, should be exposed. The pCloud static storefront itself does not require inbound access to the Kubuntu host.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Only if the local host exposes an approved reverse proxy.
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Restrict SSH to a trusted management subnet or VPN.
sudo ufw allow from <MANAGEMENT_SUBNET> to any port 22 proto tcp

sudo ufw enable
sudo ufw status verbose
```

Validate both IPv4 and IPv6 exposure:

```
sudo ss -tulpn
sudo ss -tulpn6
podman ps --format 'table {{.Names}}\t{{.Ports}}\t{{.Status}}'
```

12.2 Container security

Every service should use, where compatible:

- Rootless Podman.
- Dedicated service account.
- Read-only root filesystem.
- Explicit writable mounts only.
- No-new-privileges policy.
- Dropped Linux capabilities.
- Minimal container image.
- Pinned image digest.
- Health checks.
- CPU, memory, PID, and file-descriptor limits.
- Separate secrets.
- Separate network.
- No host network mode.
- No shared volumes across domains.
- Device mappings only where required.

12.3 Secrets

Use systemd credentials, protected environment files, or a dedicated secrets manager. Never put secrets in:

- Quadlet files.
- Git repositories.
- HTML/JavaScript.
- pCloud Public Folder.
- Corda transaction payloads.

- IPFS artifacts.
- Logs.
- Shell history.
- Documentation examples.

13. Backup, Recovery, and Updates

13.1 Backup priorities

Priority	Data
Critical	Corda node data, Corda database, ledger certificates, sales order data, receipt records
High	Field telemetry, mapping deliverables, source imagery, mission releases, simulation manifests
Medium	WebODM configuration, facility and vehicle models, container/Quadlet definitions, storefront releases
Rebuildable	Container images, caches, temporary processing files, model downloads where source/version is recorded

13.2 Backup method

- Application-aware PostgreSQL backups for each database.
- Corda-specific backup and recovery plan.
- Restic or equivalent encrypted backup system for `/ALWAYSON/config` , `/ALWAYSON/data` , `/ALWAYSON/artifacts` , and required certificate material.
- Encrypted external SSD, NAS, or remote repository.
- Encrypted pCloud replication for approved archive classes.
- Periodic restore test to an isolated location.
- Backup-age, integrity, and free-space monitoring.

13.3 Updates

Use digest-pinned images and promote releases deliberately.

Service class	Update policy
Reverse proxy and low-risk monitoring	May use controlled Podman auto-update after validation
Mastodon integration / noncritical AI worker	Scheduled update with rollback
WebODM	Staged update after backup and processing test
PostgreSQL, Redis, Corda	Manual maintenance window, backup, migration test, rollback plan
Field/RNS/MeshChatX	Manual validated release
Simulation stacks	Version-pinned for reproducibility
Drone release artifacts	Signed release only; no automatic in-field update

Quadlet is the intended declarative mechanism for Podman/systemd lifecycle management. [redhat](#)

14. Acceptance Criteria

Host and isolation

- Host reboots and all approved systemd/Quadlet services recover automatically.
- No unexpected public ports are open.

- IPv4 and IPv6 firewall validation passes.
- Containers in one domain cannot resolve, connect to, or mount data from unauthorized domains.
- No shared secret, database credential, or writable volume exists across domains.

Field and radio

- Raspberry Pi 5/Waveshare HAT and Heltec V3 communicate using a documented radio profile.
- Packets have sequence numbers, authentication, replay protection, and retry/acknowledgement behavior.
- Telemetry survives a simulated RF outage through local spool/retry behavior.
- Raw telemetry is retained locally and provenance manifests are committed through the ledger gateway.

Mapping

- Validated imagery is ingested and processed by WebODM under Podman.
- Deliverables and processing metadata are stored outside container layers.
- A completed mapping product creates a signed manifest and Corda provenance receipt.
- CPU-only operation works before optional GPU acceleration is enabled.

Vehicle simulation

- ROS 2 Jazzy/Gazebo Harmonic/ArduPilot SITL complete a representative mission.
- MAVLink stays internal to the vehicle simulation domain.
- Results are versioned, hashed, signed, and submitted only through the ledger-ingestion gateway.

Fabrication simulation

- The facility world models robot arms, storage, transfer systems, 3D printer, LPBF area, assembly, refrigerator/pantry, kitchen, carousels, and pass-throughs.
- ROS/Gazebo discovery does not cross into vehicle simulation.
- Safety zones and task interlocks are tested.
- Results produce signed, traceable provenance manifests.

Sales and community

- The pCloud HTML site exposes only static content and approved external links.
- Payment processing happens through the selected hosted provider.
- Payment webhooks are authenticated and idempotent.
- Corda records receipt/entitlement state without payment details or customer PII.
- Mastodon and AI support cannot access payment, drone, mapping, simulation, or Corda-core services.
- Public AI-generated statements require approval during the initial release phase.

Recovery

- PostgreSQL restoration passes.
- Corda recovery procedure passes in a test environment.
- A mapping dataset and simulation artifact are restored and hash-verified.
- A storefront release is rebuilt and republished to pCloud from the signed local release artifact.
- A lost/failed update rolls back to the recorded prior image digest.

15. Implementation Sequence

1. Provision the Kubuntu host, storage, firewall, Podman, Quadlet, monitoring, and encrypted backup target.
2. Create `/ALWAYS ON`, its filesystem permissions, persistent storage model, and configuration repository.
3. Create isolated Podman networks and domain-specific service accounts.

4. Establish the pCloud storefront development/build/release workflow before publishing customer-facing content.
5. Implement hosted payment checkout and verified webhook processing; create a conventional sales-order database.
6. Deploy Corda core, Corda database, and the isolated ledger-ingestion gateway.
7. Configure the Raspberry Pi 5/Waveshare radio and Heltec V3 field gateway; validate secure telemetry independently from MAVLink integration.
8. Deploy WebODM on Podman with persistent storage and CPU-only validation.
9. Deploy the vehicle simulation domain: ROS 2 Jazzy, Gazebo Harmonic, ArduPilot SITL, MAVLink, and QGroundControl simulation.
10. Deploy the separate fabrication/kitchen simulation domain with distinct ROS and Gazebo partitions.
11. Add artifact manifests, Corda provenance receipts, encrypted pCloud replication, and private/encrypted IPFS pinning.
12. Deploy Mastodon integration, OpenClaw, and LM Studio under strict sales-domain permissions and human approval workflows.
13. Run end-to-end security, failover, backup/restore, mapping, field-telemetry, sales-receipt, and simulation acceptance tests.

This architecture provides a defensible separation between public sales/community functions, private drone operations, mapping, vehicle simulation, fabrication simulation, and the ledger-backed provenance layer—while preserving the ability to connect approved results through signed manifests, Corda receipts, encrypted archival replication, and controlled product fulfillment.